

ארכיטקטורת מחשבים

הקדמה

- אריאל ביתן
- מייל - bitanar@post.bgu.ac.il
- כשפונים במייל נא לציין שם ותז שאוכל לתת מענה מתאים
- שעת קבלה ביום רביעי ב-14 - אם יהיה צורך נוסף/שעה אחרת ניתן לתאם במייל
- מילואמניקים ופטורים/הקלות נוספים כמובן לפנות במייל

שיעור ראשון

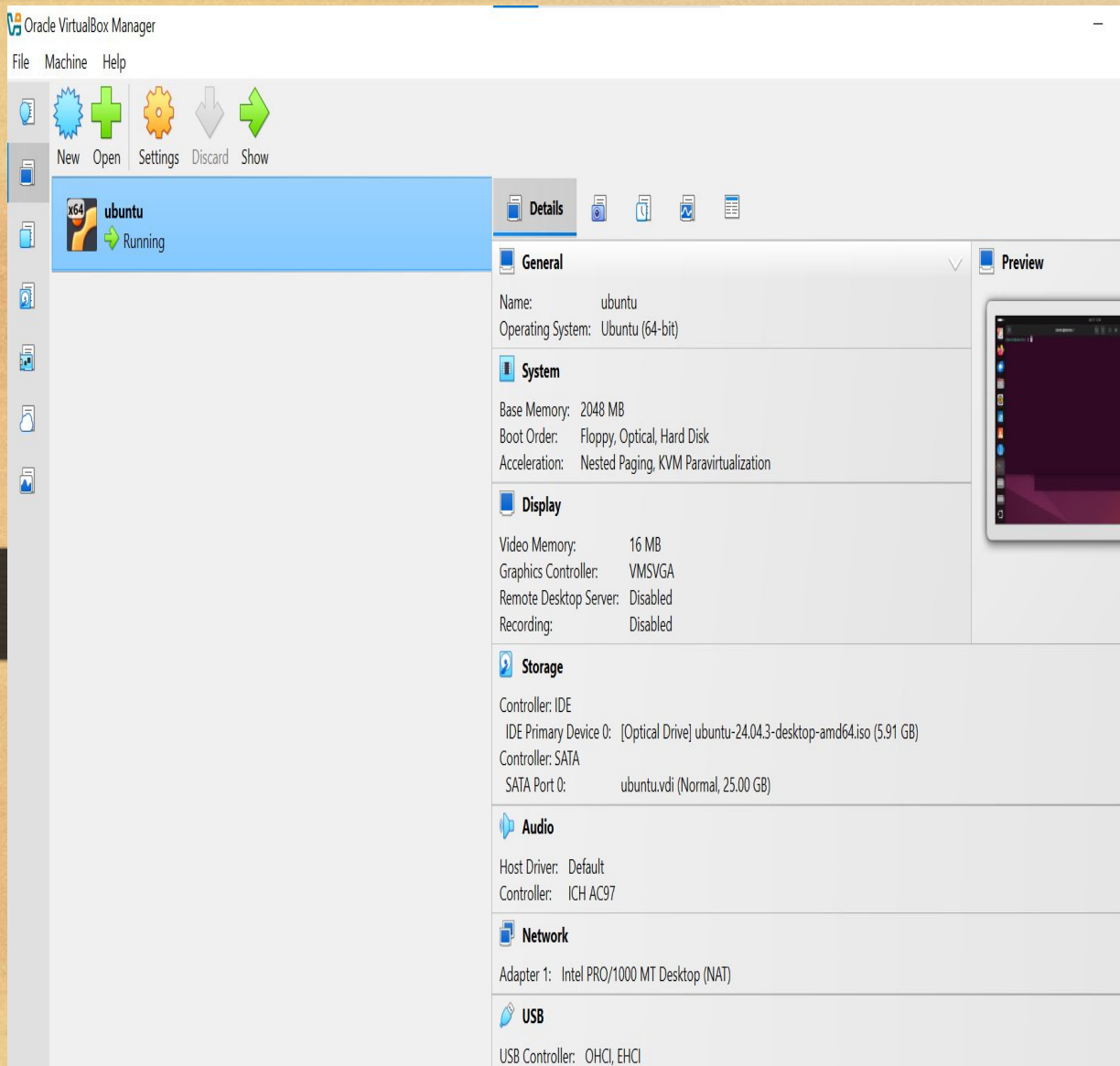
- נלמד כיצד להפעיל את המהדר של אסמבלר ב Linux
- נלמד כיצד לדבג תכניות
- נראה מספר דוגמאות של תכניות ונדגים את החומר עצמו

התקנת UBUNTU

- בקורס נעבוד עם מערכת ההפעלה של לינוקס בהפצת UBUNTU
- ניתן לעבוד עם מערכת ההפעלה הזו על ידי התקנת מכונה וירטואלית או דרך הWSL
- ממליץ לעבוד עם הWSL כי זה Built-In של ווינדוס ולכן העברת קבצים וזמן תגובה יותר מהיר ונוח

הקדמה - התקנת מכונה וירטואלית

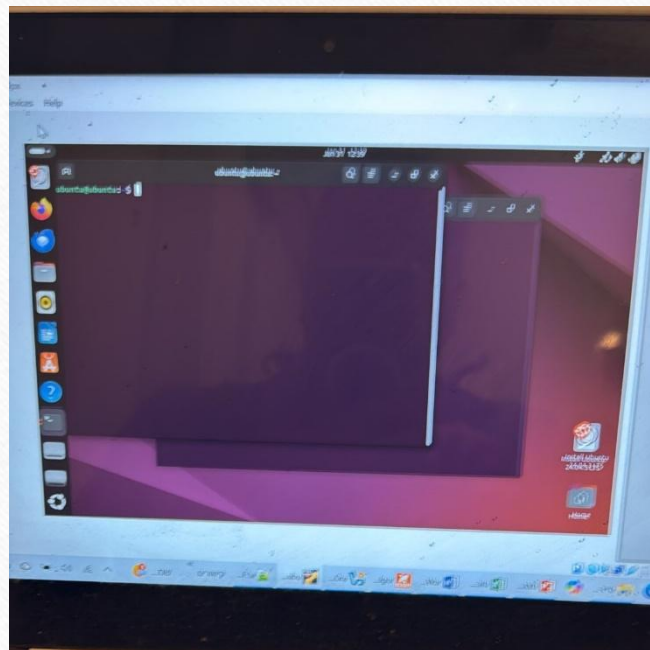
- יש להתקין על המחשב מכונה וירטואלית דרך האתר :
- <https://ubuntu.com/download/desktop>
- יש לבחור לאחר הכניסה אל הקישור באפשרות
- **Ubuntu 22.04.4 LTS** (the long-term support version)
- יש להקליק על הלחצן download ולשמור את הקובץ עם סיומת iso במקום כלשהו



המשך התקנה

- יפתח לפניכם מסך :
- יש לבחור בצד שמאל ב ubuntu
- ומשם ממשיכים לפי ההוראות
- אחת ההוראות תדרוש מכם להקליד שם
- משתמש וסיסמא. בצעו זאת.

תחילת העבודה



- לאחר שהתקנה עברה באופן תקין, יתקבל מסך כדלקמן :
- לשם תחילת העבודה יש ללחוץ על 3 מקשים בו זמנית :
- `Ctrl + Alt + T`
- כתוצאה מכך יפתח מסוף (terminal) לצורך עבודה.
- בשלב הבא – יש לבצע התקנות של כלים נדרשים

התקנת הכלים הנדרשים

- במסוף Ubuntu יש להקליד את הפקודות הבאות :

- `sudo apt update`
- `sudo apt install -y nasm gcc gdb`

- פקודות אילו מבצעות התקנה של:

- **nasm** → assembler
- **gcc** → compiler
- **gdb** → debugger

התקנה המשך

- לאחר כל פקודה כזו תראו על המסך שורות רצות. לא להתרגש מכך
- בהמשך – אם תידרשו להקליד סיסמת כניסה – בצעו זאת.

טרם תחילת עבודה

- יש ליצור קובץ בשם gdbinit וזאת כדי לקבל סביבת עבודה נוחה.
- לשם כך, מבצעים את השלבים הבאים:
- 1. `nano ~/.gdbinit` זהו קובץ טקסט עם הגדרות וסט פקודות לטרמינל לפי מילת קיצור
- 2. כותבים את הפקודות באופן ידני או לחילופין מעתיקים ומדביקים את הפקודות לפי החומר שהמרצה חילק בהרצאה/מודל

gdbinit

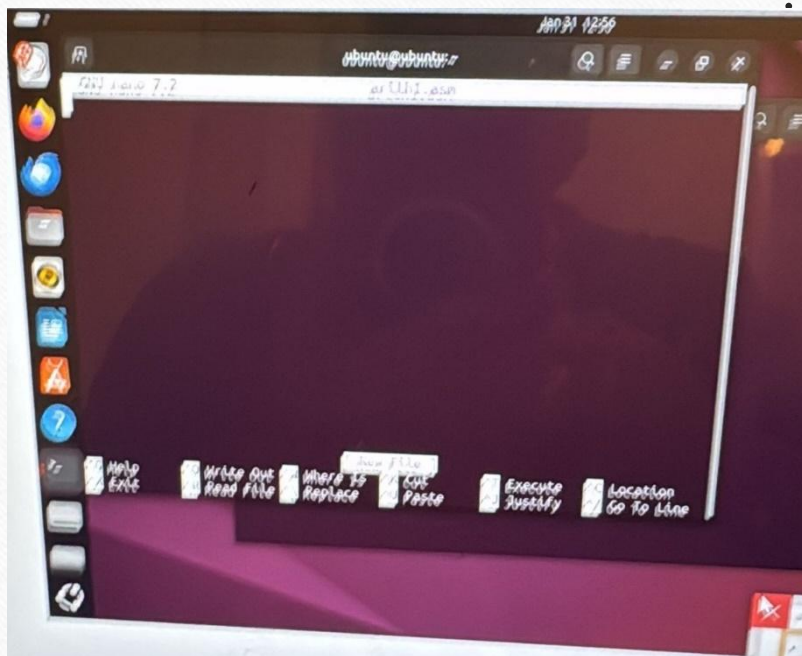
```
set debuginfo enabled on

# switch to commandline debugging
define line
  tui disable
end

# switch to debugging c:
define clang
  tui enable
  set extended -prompt clang (\w)>
  set tui border-mode standout
  set tui active-border-mode standout
  set tui border-kind acs
  layout src
end

# switch to debugging assembly - language:
define asm
  set extended -prompt asm (\w)>
  set disassembly-flavor intel
  layout asm
  tui reg all
  set tui border-mode standout
  set tui active-border-mode standout
  set tui border-kind acs
end
```


שלבי בניית תכנית אסמבלי



- יש לרשום `nano file_name.asm` ויפתח החלון הבא

- בחלון זה יש לכתוב את התכנית (אם מעתיקים אז לבצע בזהירות)

בסיום הכתיבה יש לבצע שמירה ויציאה בעזרת שתי הפקודות שלהלן:

שמירה - `Ctrl + O, Enter`

יציאה - `Ctrl + X`

המשך

- כעת, שלב ראשון בהרצה הוא להפוך את קובץ התכנית asm. לקובץ object באמצעות NASM, תוך אפשרות "לדבג" את התכנית:
 - `nasm -f elf64 file_name.asm -o file_name.o`
 - במידה ואין טעויות, לא יתקבל כל משוב מפקודה זו.
- בהמשך יש להפעיל את linker תוך שימוש בפקודה:

```
gcc file_name.o -o file_name
```

ע"י כך נקבל קובץ הרצה (במקרה שלנו במקום file_name נרשום arith1)

טיפ

- כדי לייצר קבצי ריצה לתיקייה שלמה של קבצי asm ניתן לכתוב:

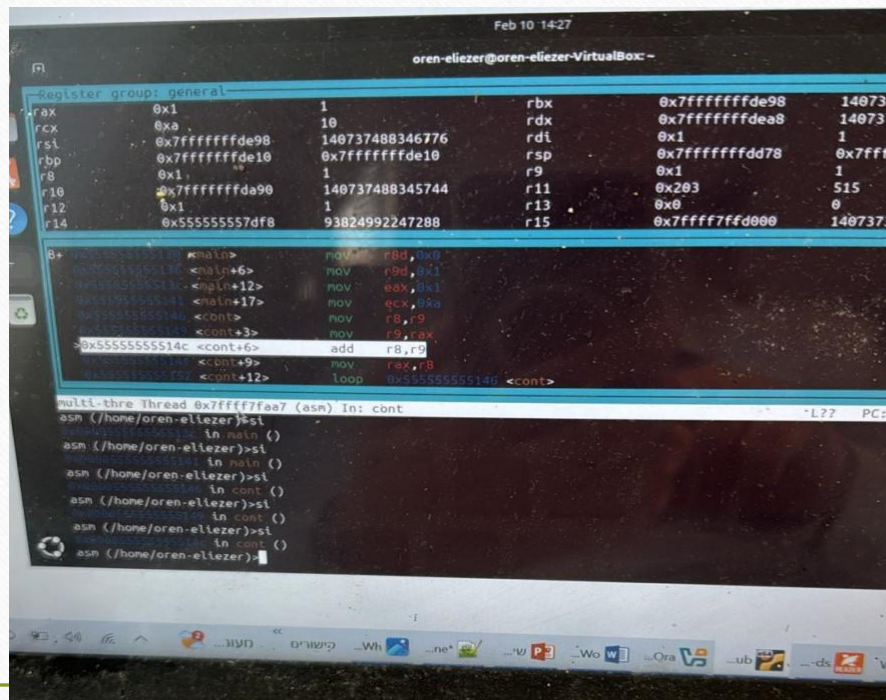
```
for f in *.asm; do name=${f%.asm}; nasm -f elf64 "$f" -o "$name.o" &&  
gcc "$name.o" -o "$name"; done
```


המשך

- כדי לבצע debugging לתכנית בעזרת gdb יש לבצע את הפקודה :
- `gdb ./file_name` וזאת ללא כל סיומת.
- שלב הבא – מוסיפים נק' `break` באופן הבא : רושמים `break main` ואז תיווצר עצירה בשורה של `main` (יש לדאוג כי קובץ התכנית יכיל תווית `main`)
- שלב הבא – רושמים `run` כדי להריץ את התכנית
- שלב הבא : רושמים `layout asm` ואז יפתח חלון של ה `debugger` של התכנית.
- כעת רושמים בחלון של ה `layout regs` : `debugger` כדי לעקוב אחר הדגלים

תמונת הדיבגר – כללית

- כפי שניתן לראות , המסך מחולק לשניים :
- החלק העליון מכיל את האוגרים והדגלים ואילו התחתון מכיל את הפקודות.

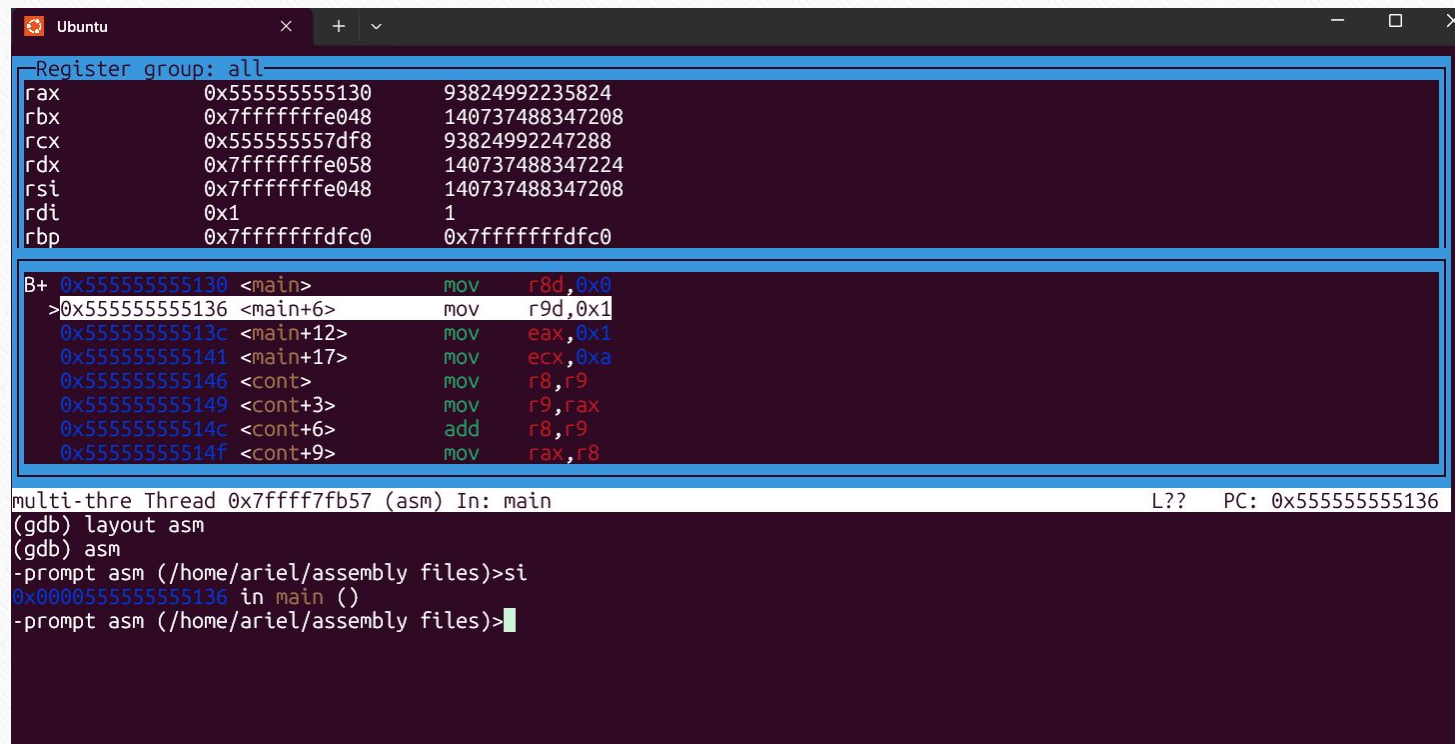


The screenshot shows a debugger window with the title bar "oren-ellezer@oren-ellezer-VirtualBox: -". The main window is divided into two panes. The top pane, titled "Register group: general", displays the state of various CPU registers (rax, rcx, rsi, rbp, r8, r10, r12, r14, rdx, rdi, rbp, r9, r11, r13, r15) with their current values and addresses. The bottom pane displays assembly code for a function named "multi-thre Thread 0x7ffff7faa7 (asm) In: cont". The code includes instructions like "mov", "add", and "loop", with comments indicating the current instruction pointer (PC) and the state of the instruction pointer (IP).

המשך

- כדי להריץ את התכנית צעד אחר צעד, רושמים `si` ואז בחלון שנפתח קודם לכן ניתן לעקוב אחר הרצת התכנית
- הערה: תוכן האוגרים יידרס ולא יוצג למסך אם אחרי הפקודה `layout regs` רושמים `layout asm`
- פקודה שימושית נוספת היא `$rip x/10i` אשר מראה את זרימת התכנית באופן ברור יותר

תמונת debugger לאחר מילת הקיצור asm שהגדרנו ב gdbinit



The screenshot shows a GDB window titled 'Ubuntu'. The top panel displays the 'Register group: all' with the following values:

Register	Value	Address
rax	0x55555555130	93824992235824
rbx	0x7fffffff048	140737488347208
rcx	0x55555555df8	93824992247288
rdx	0x7fffffff058	140737488347224
rsi	0x7fffffff048	140737488347208
rdi	0x1	1
rbp	0x7fffffffdfc0	0x7fffffffdfc0

The middle panel shows the assembly code for the current thread (multi-thre Thread 0x7ffff7fb57 (asm) In: main). The code is as follows:

```
B+ 0x55555555130 <main>      mov     r8d,0x0
>0x55555555136 <main+6>    mov     r9d,0x1
0x5555555513c <main+12>    mov     eax,0x1
0x55555555141 <main+17>    mov     ecx,0xa
0x55555555146 <cont>      mov     r8,r9
0x55555555149 <cont+3>    mov     r9,rax
0x5555555514c <cont+6>    add     r8,r9
0x5555555514f <cont+9>    mov     rax,r8
```

The bottom panel shows the GDB prompt and the current state of the thread:

```
(gdb) layout asm
(gdb) asm
-prompt asm (/home/ariel/assembly files)>si
0x000055555555136 in main ()
-prompt asm (/home/ariel/assembly files)>
```

The status bar at the bottom indicates the current thread is 'multi-thre Thread 0x7ffff7fb57 (asm) In: main' and the PC is '0x55555555136'.

שמירה והצגת הקובץ

- כדי להציג את הקובץ לאחר שנכתב ב ubuntu, יש לצאת מה debugger כלומר לצאת מ gcc ע"י הקלדה של `exit`, ואז רושמים `cat file_name` עם הסיומת `.asm`.
- לאחר מכן, סימון של התכנית עם העכבר וכן לחיצה על העכבר הימני תוביל אתכם לתפריט שבו אחת האופציות תהיה `copy`. כעת ניתן יהיה להעתיק את התכנית אל `notepad` (הערה – לא תמיד אפשרות זו פעילה ואז יש להעתיק את הפקודה שורה אחר שורה דהיינו לכתוב כל שורה ידנית).
- כדי לאתר את הקובץ ואת מיקומו, יוצאים מה debugger ורושמים `pwd` ואז מקבלים את שם התיקייה שבה נמצא הקובץ.
- כדי לאתר את כל הקבצים בתיקייה הזו רושמים `ls` ושם ניתן למצוא את כל הקבצים, ביניהם גם את קובץ התכנית הספציפית.

תוכן הדגלים בתכנית

Flag	Name	Set?	Meaning
ZF	Zero Flag	0	Result is not zero ($10 \neq 20$)
SF	Sign Flag	1	Result is negative (-10)
CF	Carry Flag	1	Borrow occurred → unsigned comparison: $10 < 20$
OF	Overflow Flag	0	No signed overflow occurred
PF	Parity Flag	?	Depends on the parity of the low byte of the result (not critical now)

דגלים המשך

- כדי לעקוב אחר תכני הדגלים, יש לרדת למטה בחלון האוגרים ובין האוגרים ניתן יהיה להבחין באוגר הדגלים.

- או לחלופין, לכתוב `i r eflags`

```
-prompt asm (/home/ariel/assembly files)>i r eflags  
eflags          0x246          [ PF ZF IF ]
```

מה שרשום בתוך הסוגריים המרובעות זה הביטים הדלוקים (שערך הביט שלהם הוא 1)

conditional

תוכנית עם תנאים

- תכנית זו מאתחלת שני אוגרים `rax`, `rbx`, בודקת איזה מהם מכיל תוכן גדול יותר ומדפיסה את היחס ביניהם.
- נשמרה תחת השם `conditional.asm`
- התכנית לא תדפיס דבר ולכן יש לדבג אותה שלב אחר שלב.
- התכנית בודקת את היחס בין תכני האוגרים ע"י פקודת השוואה `cmp` אשר מבצעת חיסור בין התוכן של `rax` לתוכן של `rbx` ומשפיעה אך ורק על הדגלים. תוצאת החיסור לא נשמרת כלל.

```
-prompt asm (/home/ariel/assembly files)>i r eflags  
eflags          0x287          [ CF PF SF IF ]
```

תוכן הדגלים -

- הסבר : בתכנית rax מכיל ערך נמוך יותר מ rbx ולכן ברגע שמחסרים התוצאה שלילית.
- על כן , התוצאה שונה מאפס ולכן $zf=0$
- דגל הסימן $sf=1$ כי התוצאה שלילית.
- דגל הנשא $cf=1$ כי נוצר לווה – borrow
- דגל הגלישה $of=0$ כי אין גלישה
- דגל הזוגיות pf תלוי במספר האחדים בתוצאה.
- בכל אופן אוגר הדגלים מראה 0x287 בהתאם לדגלים הנ"ל

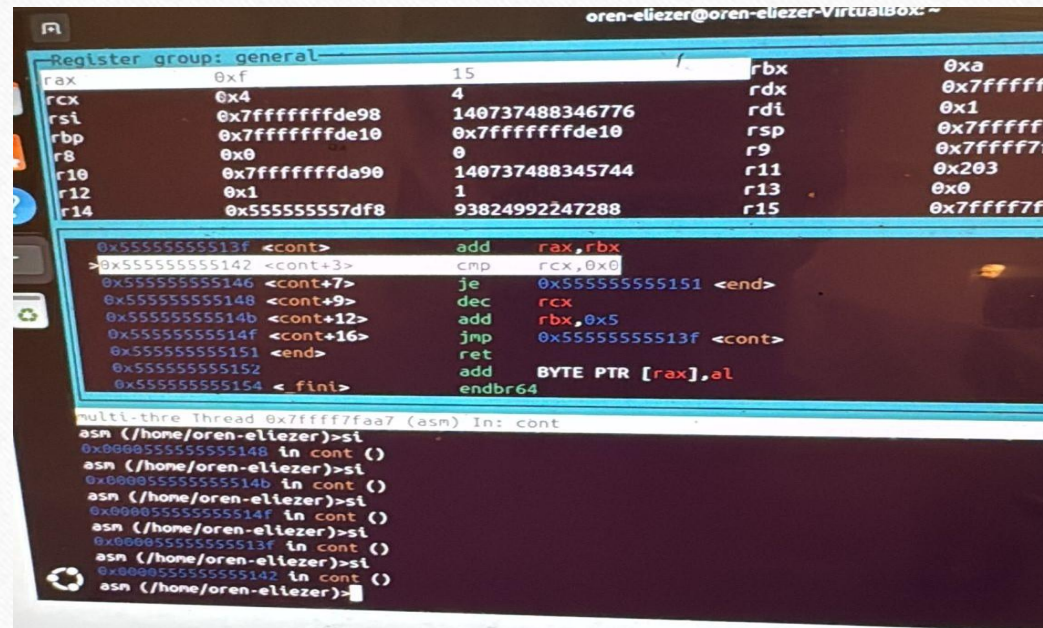
sumjump

תכנית הסוכמת 5 מספרים תוך שימוש בלולאה

- התכנית מריצה לולאה תוך שימוש בפקודה `jmp` ומחשבת את הסכום $5+10+15+20+25$
- לשם כך מגדירים את `rax` כאוגר מסכם, את `rbx` אשר יכיל בתחילה את 5 ויקודם בכל שלב בלולאה ב - 5, ואת `rcx` כמונה של הלולאה – לכן מאותחל ב 5.
- בכל שלב בלולאה האוגר `rbx` מקודם ב 5 ואילו `rax` משמש כמסכם. כמו כן מונה הלולאה `rcx` יורד ב 1
- בכל שלב בלולאה מתבצעת השוואה בין תוכנו של `rcx` ל 0 שהוא תנאי הסיום ללולאה.

תמונת הדיבגר לאחר שני שלבים

- ניתן לראות את תמונת ה debugger לאחר שני שלבי לולאה :



The screenshot shows a debugger window with the title bar "oren-eliezer@oren-eliezer-VirtualBox: ~". The main window is divided into two panes. The top pane, titled "Register group: general", displays the values of 16 registers (rax, rcx, rsi, rbp, r8, r10, r12, r14, rbx, rdx, rdi, rsp, r9, r11, r13, r15) in hexadecimal. The bottom pane displays assembly code with addresses and instructions. The instructions include "add rax,rbx", "cmp rcx,0x0", "je 0x55555555151 <end>", "dec rcx", "add rbx,0x5", "jmp 0x5555555513f <cont>", "ret", "add BYTE PTR [rax],al", and "endbr64". Below the assembly code, there is a command prompt showing the execution of "asm (/home/oren-eliezer)>si" and the resulting address "0x000055555555148 in cont ()".

```
Register group: general
rax 0xf 15 rbx 0xa
rcx 0x4 4 rdx 0x7fffffff
rsi 0x7fffffffde98 140737488346776 rdi 0x1
rbp 0x7fffffffde10 0x7fffffffde10 rsp 0x7fffffff
r8 0x0 0 r9 0x7fffffff
r10 0x7fffffffda90 140737488345744 r11 0x203
r12 0x1 1 r13 0x0
r14 0x555555557df8 93824992247288 r15 0x7fffffff

0x5555555513f <cont> add rax,rbx
>0x55555555142 <cont+3> cmp rcx,0x0
0x55555555146 <cont+7> je 0x55555555151 <end>
0x55555555148 <cont+9> dec rcx
0x5555555514b <cont+12> add rbx,0x5
0x5555555514f <cont+16> jmp 0x5555555513f <cont>
0x55555555151 <end> ret
0x55555555152 add BYTE PTR [rax],al
0x55555555154 <fini> endbr64

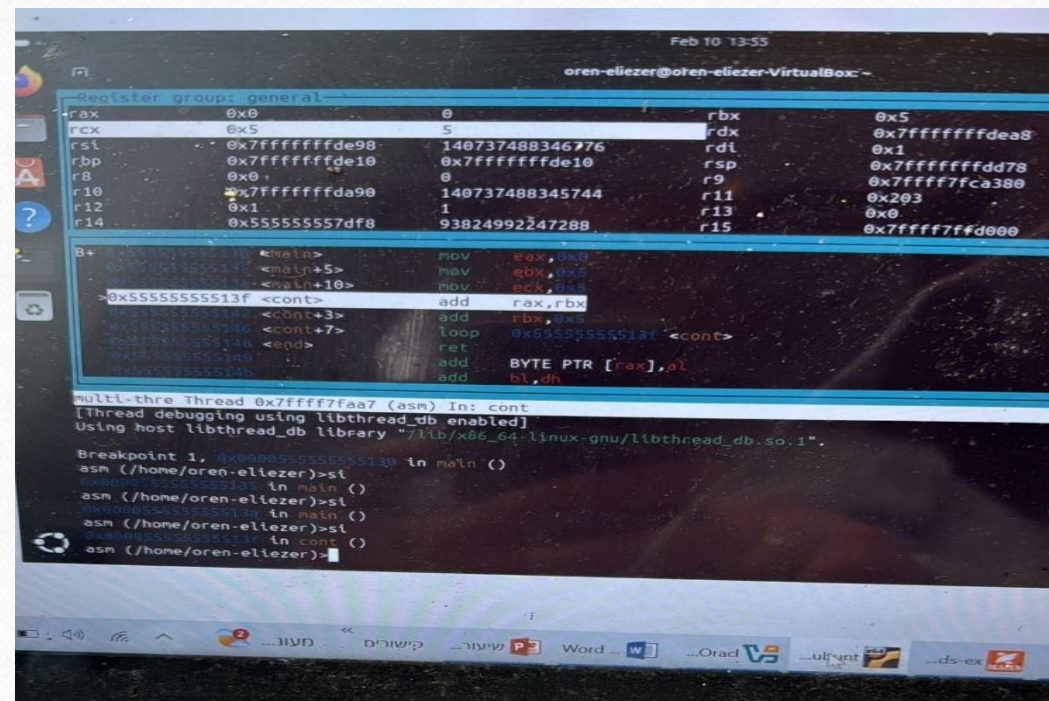
multi-thre Thread 0x7ffff7faa7 (asm) In: cont
asm (/home/oren-eliezer)>si
0x000055555555148 in cont ()
asm (/home/oren-eliezer)>si
0x00005555555514b in cont ()
asm (/home/oren-eliezer)>si
0x00005555555514f in cont ()
asm (/home/oren-eliezer)>si
0x00005555555513f in cont ()
asm (/home/oren-eliezer)>si
0x000055555555142 in cont ()
asm (/home/oren-eliezer)>
```


sumloop

גרסה נוספת לאותה תכנית העושה שימוש בפקודת loop

- בתכנית זו בוצע שימוש בפקודת LOOP.
- שימוש בפקודת loop מקטינה את האוגר rcx ב 1 ומשווה לאפס. כל עוד rcx שונה מאפס, הלולאה תמשיך כרגיל.
- בלולאה עצמה מבצעים : 1. סיכום תוכנו של rbx עם rax
- 2. קידומו של rbx ב 5

תמונת דיבגר



The screenshot shows a debugger window with the following content:

Feb 10 13:55
oren-ellezer@oren-ellezer-VirtualBox: ~

Register group: general

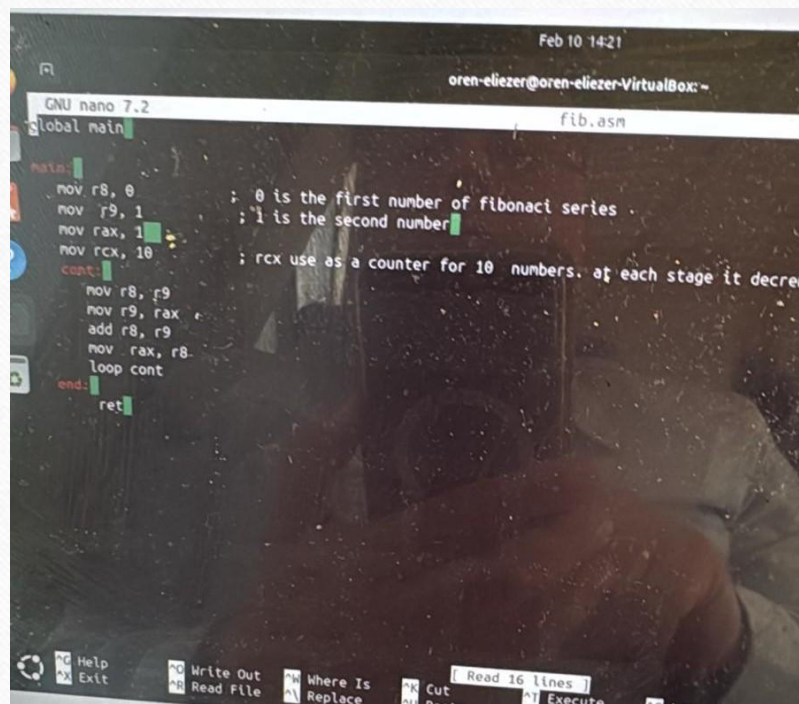
Register	Value	Register	Value
rax	0x0	rbx	0x5
rcx	0x5	rdx	0x7fffffffdea8
rsi	0x7fffffffde98	rdi	0x1
rbp	0x7fffffffde10	rsp	0x7fffffffdd78
r8	0x0	r9	0x7fffffffca380
r10	0x7fffffffda90	r11	0x203
r12	0x1	r13	0x0
r14	0x5555555555df8	r15	0x7ffff7fd000

8+ ... <main> mov eax, 0x0
0x5555555555140 <main+5> mov ebx, 0x5
0x5555555555144 <main+10> mov ecx, 0x1
0x555555555513f <cont> add rax, rbx
0x5555555555143 <cont+3> add rdx, rdx
0x5555555555146 <cont+7> loop 0x555555555514f <cont>
0x5555555555148 <epd> ret
0x555555555514b add BYTE PTR [rax], al
0x555555555514c add bl, dh

Multi-thread Thread 0x7ffff7faa7 (asm) in: cont
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, 0x555555555513f in main ()
asm (/home/oren-ellezer)>sl
0x5555555555140 in main ()
asm (/home/oren-ellezer)>sl
0x5555555555143 in main ()
asm (/home/oren-ellezer)>sl
0x555555555514f in cont
asm (/home/oren-ellezer)>

סדרת פיבונצ'י



```
Feb 10 14:21
oren-eliezer@oren-eliezer-VirtualBox:~
GNU nano 7.2 fib.asm
global main

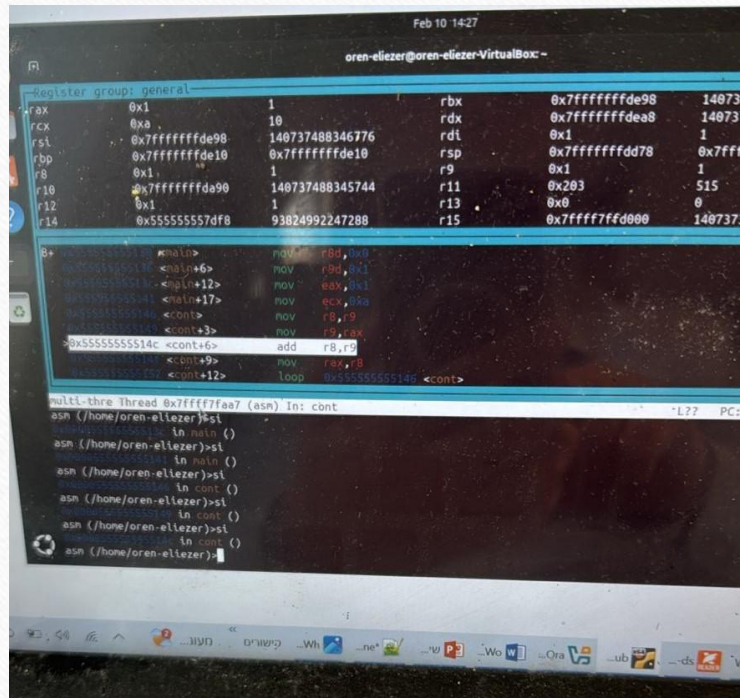
main:
    mov r8, 0      ; 0 is the first number of fibonacci series
    mov r9, 1      ; 1 is the second number
    mov rax, 1
    mov rcx, 10    ; rcx use as a counter for 10 numbers. at each stage it decre
cont:
    mov r8, r9
    mov r9, rax
    add r8, r9
    mov rax, r8
    loop cont
end:
    ret
```


fib

סדרת פיבונצ'י

- השתמשתי באוגרים כלליים r8, r9 וכן ב rax על מנת לחשב את הערך השלישי בסדרה.
- העקרון הוא לאתחל את r8 ב 0, את r9 ב 1, לסכם אותם ואת הסכום לאחסן ב rax.
- בכל שלב בלולאה תוכנו של r9 עובר ל r8, תוכנו של rax עובר אל r9 ואז הסכום של r8 עם r9 עובר ל rax.
- התכנית מחשבת את כל הערכים בסדרה עד לאיבר העשירי.

Debugger



פקודות כפל

- באסמבלר קיימות שתי `MUL`, `IMUL`
- `MUL` משמשת לכפל של מספרים לא מסומנים והאחרת משמשת לכפל של מספרים מסומנים.
- דוגמא :
- `mov rax, 6`
- `mov rbx, 7`
- `mul rbx`

פקודות כפל המשך

- מה מבצעת MUL ? היא כופלת את אוגר rax באופרנד אשר יכול להיות מספר או אוגר אחר , ואת התוצאה שומרת בזוג אוגרים : rdx:rax , כלומר נשמר מקום של 128 ביטים לתוצאה.
- בדוגמא האחרונה כופלים 6 ב 7 ומקבלים 42 אשר נשמר ב rax והיות והתוצאה איננה חורגת מ 64 ביט , היא נשמרת ב rax בעוד ש rdx מאופס.
- פקודת IMUL פועלת באופן דומה אלא שהכפל נעשה באמצעות כפל של מספרים מסומנים.

חישוב עצרת של מספר

- לצורך חישוב עצרת של מספר אין צורך בגרסה של מספרים מסומנים ולכן נשתמש ב .MUL
- נדגים להלן תכנית אשר מחשבת את 10!

factorial

תכנית לחישוב עצרת

```
; factorial using MUL for calculating 10!

global main

main:
    mov rcx, 10          ; rcx contains 10 in order to get 10!
    mov rax, 1           ; rax is the accumulator
    cqo
cont:
    cmp rcx, 1
    jz done
    mul rcx
    loop cont

done:
    ret
```

הסבר

- כדי לחשב 10! נאחסן את המספר 10 באוגר rcx ונאתחל את rax בערך של 1 כדי ש rax ישמש ככופל בכל אחד מן השלבים.
- הכפל יתבצע ע"י הפקודה mul rcx אשר כופלת את תוכנו של rax בתוכנו של rcx
- חשוב טרם ביצוע הכפל ולפני הכניסה ללולאה לרשום את הפקודה cqo. פקודה זו מנקה את rdx ומאפסת אותו וזאת על מנת לקבל תוצאה אמיתית.
- בשלב הבא תוכנו של rcx ירד ב 1 ובכל שלב תתבצע השוואה בין תוכנו של rcx לאפס לצורך סיום הלולאה.
- למרות שהתוצאה גבוהה – מעל 3 מיליון, היות והאוגרים שאנו עובדים איתם הינם בני 64 ביט, התוצאה תישמר ב rax בעוד ש rdx מאופס.